

Hoja de ruta del día

- **Módulo:** Módulo 3: Machine Learning
- **Semana / Sesión:** Semana 15 - Fase 1
- **Temática Central:** Árboles de Decisión y Ensembles (Random Forest / Boosting): primer entrenamiento de un modelo de clasificación
- **Stack Requerido:** Python, Scikit-Learn, Pandas, Matplotlib/Seaborn (opcional para visualizar el árbol)

1. Objetivos de aprendizaje del día

Al finalizar la jornada de investigación autónoma, debes ser capaz de:

- **Objetivo 1:** Entrenar un `DecisionTreeClassifier` y un `RandomForestClassifier` con Scikit-Learn sobre un dataset simple, entendiendo qué hace cada hiperparámetro básico (`max_depth`, `criterion`, `n_estimators`).
- **Objetivo 2:** Identificar por qué un árbol de decisión sin restricciones tiende al overfitting, y cómo Random Forest mitiga ese problema combinando varios árboles entrenados con bagging.
- **Objetivo 3:** Evaluar el modelo entrenado con métricas básicas de clasificación (accuracy, matriz de confusión) y comparar el rendimiento del árbol individual contra el ensemble.

2. Bibliografía

Deberás investigar de forma independiente las siguientes fuentes de referencia para solucionar el reto técnico.

- **Documentación Oficial 1:**
 - Clase `DecisionTreeClassifier` (parámetros y API completa): <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>
 - Guía conceptual "1.10. Decision Trees" (teoría, criterios de split, ejemplos): <https://scikit-learn.org/stable/modules/tree.html>

- **Documentación Oficial 2:** Scikit-Learn API — sección RandomForestClassifier, prestando atención a n_estimators, max_features y al atributo feature_importances_.
 - Clase RandomForestClassifier (parámetros como n_estimators, max_features, feature_importances_): <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>
 - Guía conceptual "1.11. Ensembles" (bagging, random forest, boosting, todo explicado junto): <https://scikit-learn.org/stable/modules/ensemble.html>
- **Documentación Oficial 3:** Scikit-Learn User Guide — apartado "Ensemble methods", sección de Bagging, para entender de dónde sale el comportamiento de Random Forest.
 - Clase BaggingClassifier (incluye oob_score, bootstrap explícito): <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.BaggingClassifier.html>

3. Ejemplo de implementación práctica

Fragmento que ilustra el patrón de carga, split y entrenamiento básico, sin resolver el reto:

```
python

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# X, y ya cargados desde un DataFrame de pandas
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

modelo = DecisionTreeClassifier(max_depth=3, random_state=42)
modelo.fit(X_train, y_train)

predicciones = modelo.predict(X_test)
print("Accuracy:", accuracy_score(y_test, predicciones))
```

4. Reto práctico de código:

El enunciado incluirá:

- **Contexto del problema:** DataX S.L necesita un primer modelo de clasificación para predecir si un cliente abandonará el servicio (churn) a partir de un dataset con variables como antigüedad, tipo de contrato, gasto mensual y número de incidencias soportadas. El equipo pide entrenar primero un árbol de decisión simple, y después comparar si un Random Forest mejora el resultado.
- **Componente de incertidumbre:** el dataset incluye variables categóricas y numéricas mezcladas sin preprocesar, y el coder deberá investigar por su cuenta cómo decidir la profundidad adecuada del árbol (`max_depth`) para evitar tanto el underfitting como el overfitting, ya que no se le dará un valor predefinido — deberá comparar al menos dos profundidades distintas y justificar con qué métrica decide cuál es mejor.
- **Restricción ética y responsabilidad:** el dataset tiene un desbalance entre clientes que se quedan y los que abandonan (la clase "churn" es minoritaria). El coder debe responder explícitamente: ¿qué problema genera ese desbalance al medir solo accuracy?, y ¿qué consecuencia tendría en la práctica que el modelo prediga sistemáticamente "no abandona" para todos los clientes, incluyendo aquellos que sí lo harán?